Service Bearer Tokens

Explore how bearer tokens enable secure and temporary authentication between AWS services. Understand token issuance, validation, and best practices for handling tokens in applications like Amazon Cognito, API Gateway, and EKS. This lesson helps you build secure, scalable AWS solutions using bearer token workflows.

Bearer tokens are temporary credentials commonly used for authenticating service-to-service communication. In AWS, bearer tokens play a central role in enabling secure access to services without relying on long-lived IAM credentials. They're included in requests using an `Authorization` header, and any entity in possession of the token is granted access to the specified resource, therefore, the term "bearer."

Bearer tokens work on the principle of possession, whoever holds the token can access the associated resource. Because of this, systems must enforce HTTPS and strict token handling policies.

These tokens are commonly issued by identity providers like Amazon Cognito or external OAuth providers and include claims such as the issuer (`iss`), audience (`aud`), subject (`sub`), and expiration (`exp`). These claims help validate and scope the token.

How AWS services implement bearer tokens

AWS services integrate bearer tokens in different ways depending on the c

Ask

of use. Here are some of the most common implementations:

1. Amazon Cognito issues JWTs (ID, access, and refresh tokens) during user authentication. These JWTs are bearer tokens that allow access to APIs or to AWS services like API Gateway.
2. Amazon EKS uses IAM roles for service accounts (IRSA) to grant pods access to AWS services. A projected token from the Kubernetes API serves as a bearer token, which is then validated by AWS STS.
3. IAM Roles Anywhere allows workloads outside AWS to request short-lived credentials using X.509 certificates. The request involves a bearer token signed with the private key of a trusted certificate.
4. AWS Security Token Service (STS) enables federated access by exchanging tokens from an external identity provider for temporary AWS credentials. The original token acts as a bearer token during this exchange.

Once we have a token, we pass it to other services in API requests:

```
Authorization: Bearer eyJraWQiOiJrMWQiLCJhbGciOiJSUzI1NiJ9...
```

Example of using a bearer token in the HTTP header

Bearer tokens often include:

- **Scope:** The permissions or operations the token allows (e.g., `read:userinfo`)
- **Audience (aud):** Which resource or API the token is meant for
- **Issuer (iss):** The trusted authority that generated the token
- **Expiration (exp):** Typically expires within 15 minutes to 1 hour

Tokens can be introspected or verified locally depending on whether they are opaque or structured, like JWT. AWS typically supports JWT validation without requiring a callback to the issuer.

Ask

How developers work with bearer tokens

For developers building with AWS, bearer tokens show up frequently—whether we're securing API Gateway endpoints, calling external APIs from Lambda, or implementing federated identity. We often need to extract bearer tokens from identity flows, add them to HTTP headers in client or backend requests, and validate them in our application logic.

In practice, our applications must handle the authentication flow by processing callbacks, exchanging authorization codes for tokens, and securely storing those tokens. We often rely on libraries such as `jsonwebtoken` (Node.js), `PyJWT` (Python), or `aws-jwt-verify` to decode and validate token claims.

Getting this right is essential for protecting our services while keeping access seamless and scalable. Here is a Python code example that shows how bearer tokens are used in practice:

main.py boto3

```
import requests

token = "eyJraWQiOiJrMWQiLCJhbGciOiJSUzI1NiJ9..."  # Cognito JWT
headers = {"Authorization": f"Bearer {token}"}

response = requests.get("https://<your-api-id>.execute-api.us-west-2.a

if response.status_code == 200:
    print(response.json())
else:
    print("Request failed:", response.status_code)
```

Sample code

Ask

These calls simulate an application (like a mobile frontend) making authenticated requests to an API Gateway endpoint using a Cognito JWT token as a bearer token.

Comparing bearer tokens and SigV4

While bearer tokens are widely used in federated identity and web-based applications, AWS also uses SigV4, a cryptographic request signing method. The comparison below highlights key differences:

Feature	Bearer Tokens	SigV4 Authentication
Format	Simple token string in header	Cryptographic signature in header/query
Use Case	OAuth flows, third-party APIs	AWS SDK, CLI, and direct service calls
Expiration	Short-lived	Controlled via session config

SigV4 is AWS's native request signing method, commonly used with SDKs and the CLI, unlike bearer tokens which are used in federated and third-party contexts.

Common use cases

We use bearer tokens in a variety of scenarios where we need to avoid static credentials and support secure delegation:

- Lambda functions calling third-party APIs requiring OAuth tokens
- Applications using Amazon Cognito for authentication and authorization
- Amazon EKS workloads leveraging IAM roles for service accounts (IRSA)
- External workloads accessing AWS services through IAM Roles Anywhere
- Applications interacting with services like Amazon Bedrock or Amazon Titan
- Custom authorizers in Amazon API Gateway validating tokens before granting access

Ask

- Event-driven workflows using Step Functions with HTTP tasks requiring token-based auth

These examples show how bearer tokens serve as the bridge between identity and access in modern cloud architectures.

For developers, bearer tokens are essential when building secure web and mobile applications that interact with APIs. Whether it's securing endpoints in API Gateway with Cognito JWTs, configuring authentication in Lambda function handlers, or handling OAuth flows with external identity providers, developers need to know how to acquire, validate, and forward tokens correctly in code.

Security considerations

Because bearer tokens grant access to resources, we must take the same precautions with them as we do with passwords or access keys:

- Store them securely using AWS Secrets Manager or Systems Manager Parameter Store.
- Never expose them in logs, URLs, or error messages.
- Always use HTTPS to prevent interception.
- Rotate or refresh tokens regularly to maintain secure sessions.
- Use audience and issuer validation to ensure tokens are accepted only from and for trusted sources.
- Implement token revocation or short TTLs to reduce risk in the event of a compromise.

Security best practices help us reduce the risk of credential leaks and unauthorized access.

Ask

Conclusion

Bearer tokens are a fundamental mechanism in AWS for enabling secure, temporary, and delegated access across multiple services without relying on long-lived credentials. They play a critical role in authentication and authorization workflows involving Amazon Cognito, API Gateway, EKS IRSA, STS federated access, and IAM Roles Anywhere. Understanding how these tokens are issued, passed in requests, validated, and exchanged for temporary credentials is essential for building secure applications.

Ask